

TC2038 - Análisis y diseño de algoritmos avanzados

Algoritmos de la mochila y del viajero

Profesor - Alison Muñoz Capote
Computación - Facultad de Ingeniería

Mapa del Tema y Objetivos

Objetivo de la Sesión:

Comprender y modelar problemas clásicos de **Optimización Combinatoria y Complejidad NP**, dominando su resolución mediante **Programación Dinámica** y **Árboles de Espacio de Estados**.

Índice:

1. El Problema de la Mochila: Motivación y Teoría con PD
2. Mochila 0/1: Ejemplo Guiado y Reto en el Aula
3. ¿Por qué la Mochila está en el bloque de Grafos? (DAG y *Branch & Bound*)
4. Implementación en Python (Mochila 0/1 + *Backtracking*)
5. El Problema del Viajante (TSP) y la Frontera **P vs. NP**
6. TSP en el Pizarrón: *Branch & Bound* con Cotas Inferiores y Heurísticas
7. Asignación de Estudio Independiente

1. El Problema de la Mochila

El Dilema del Dron de Rescate

Situación Problema

Eres el Ingeniero de Vuelo de una brigada de rescate en alta montaña. Un dron autónomo debe llevar suministros médicos críticos a un campamento aislado.

- La batería y los motores del dron soportan una carga útil **máxima estricta de** $W = 5\ \text{kg}$.
- Tienes n cajas de suministros; cada una tiene un **peso (w_i)** y un **valor de impacto vital (v_i)**.
- **Misión:** Maximizar el valor total transportado sin superar los $5\ \text{kg}$ (un gramo extra impide el despegue).

Dos Mundos Distintos: Fraccionaria vs. 0/1

Antes de programar, un ingeniero debe identificar qué tipo de carga está manipulando:

Variante	Regla Principal	Estrategia Óptima	Complejidad
Mochila Fraccionaria	Puedes partir los objetos (ej. llevar medio saco de grano o combustible líquido).	**Algoritmo *Greedy* (Voraz):** Ordenar por ratio valor/peso (v_i/w_i) y llenar hasta topar.	$\mathcal{O}(n \log n)$ *(Polinomial — Clase P)*
Mochila 0/1 (Entera)	**Todo o nada:** O llevas el equipo entero (1) o lo dejas (0). No puedes partir un desfibrilador a la mitad.	**Programación Dinámica** o **Branch & Bound**.	$\mathcal{O}(n \cdot W)$ *(NP-Completo)*

¿Por qué fracasa el enfoque *Greedy* en la Mochila 0/1?

La Trampa de la Decisión Voraz

Imagina una mochila de $4\ \text{kg}$ y tres objetos:

- **Objeto A:** $1\ \text{kg}$ ($\$1{,}500 \rightarrow \text{Ratio } \$1{,}500/\text{kg}$)
- **Objeto B:** $3\ \text{kg}$ ($\$2{,}000 \rightarrow \text{Ratio } \$666/\text{kg}$)
- **Objeto C:** $4\ \text{kg}$ ($\$3{,}000 \rightarrow \text{Ratio } \$750/\text{kg}$)

Si eliges por **mayor valor absoluto** (C), ganas $\$3{,}000$ y llenas la mochila.
Si eliges por **mejor ratio** (A), ocupas $1\ \text{kg}$, te sobran $3\ \text{kg}$ donde ya no cabe C, y solo si combinas **A + B** logras el verdadero óptimo: $\$3{,}500$.

En la Mochila 0/1, elegir lo mejor hoy puede dejar huecos inútiles mañana.

La Base Teórica: Programación Dinámica (PD)

Divide, Anota y Vencerás

En lugar de resolver de golpe una mochila de capacidad W, resolvemos *mini-mochilas* desde capacidad $0, 1, 2 \ldots$ hasta W, habilitando los objetos uno por uno en una matriz $DP[i][w]$.

La Ecuación de Recurrencia de Bellman:

$$DP[i][w] = \max\Big(\underbrace{DP[i-1][w]}_{\text{Opción A: NO llevar objeto } i} \quad , \quad \underbrace{v_i + DP[i-1][w-w_i]}_{\text{Opción B: SÍ llevarlo (si } w_i \leq w)} \Big)$$

Las 3 Preguntas para llenar cualquier celda

1. ¿Cabe en la mochila actual?

¿El peso del objeto wi es menor o igual a la columna actual w? Si NO cabe, copiamos el valor de arriba: DP[i-1][w].

➔

2. ¿Conviene meterlo?

Si SÍ cabe, comparamos no llevarlo (mirar arriba) vs. llevarlo (vi + mejor valor del espacio libre DP[i-1][w-wi]).

➔

3. Anotar el Ganador

Escribimos el máximo de ambos en DP[i][w] y jamás volvemos a recalcular ese subproblema.

Regla visual rápida: *Mirar una fila arriba* = No llevarlo. *Mirar una fila arriba y* w_i *columnas a la izquierda* = Espacio libre tras llevarlo.

Ejemplo 1 (Guiado): Mochila de $W = 5\ \text{kg}$

Vamos a resolver paso a paso una mochila con capacidad máxima $W = 5\ \text{kg}$ y estos 3 artículos:

Objeto (i)	Nombre	Peso (w_i)	Valor (v_i)
1	**Libro**	$1\ \text{kg}$	$\$15$
2	**Cámara**	$3\ \text{kg}$	$\$20$
3	**Consola**	$4\ \text{kg}$	$\$30$

Caso Base (Fila 0 y Columna 0): Sin objetos ($i = 0$) o sin capacidad ($w = 0$), la ganancia máxima siempre es $\$0$.

Ejemplo 1: Evaluando Fila 1 (Libro) y Fila 2 (Cámara)

- **Fila 1 — Libro ($1\text{ kg}, \$15$):** Cabe desde $w = 1\text{ kg}$. Como es el único objeto disponible hasta ahora, todas las capacidades de 1 a 5 kg valen 15.
- **Fila 2 — Cámara ($3\text{ kg}, \$20$):**
  - En $w = 1, 2\text{ kg}$: No cabe ($3 > w$). Copiamos el 15 de arriba.
  - En $w = 3\text{ kg}$: Cabe exacta. $\max(\text{arriba} = 15,\ 20 + DP[1][0]) = \mathbf{20}$.
  - En $w = 4\text{ kg}$ **(¡La magia de PD!):** Si meto la Cámara ($3\text{ kg}, \$20$), **me sobra** 1 kg **libre**. Miro la fila de arriba en 1 kg ($DP[1][1] = 15$, el Libro). Total: $20 + 15 = \mathbf{35}$.

Objeto \ Capacidad (w)	0 kg	1 kg	2 kg	3 kg	4 kg	5 kg
0. Base (Sin objetos)	0	0	0	0	0	0
1. Libro (1 kg, $15)	0	**15**	**15**	**15**	**15**	**15**
2. Cámara (3 kg, $20)	0	15	15	**20**	**35**	**35**

Ejemplo 1: Evaluando Fila 3 (Consola) y Solución

- **Fila 3 — Consola ($4\ \text{kg}, \$30$):**
  - En $w = 1, 2, 3\ \text{kg}$: No cabe. Copiamos de arriba: $15, 15, 20$.
  - En $w = 4\ \text{kg}$ **(Duelo clave):** No meterla (arriba) da $\$35$ (Cámara+Libro). Meter la Consola sola da $\$30 + DP[2][0] = \$30$. ¡Gana no meterla ($35$)!
  - En $w = 5\ \text{kg}$ **(Meta final):** Meter la Consola ($\$30$) deja $1\ \text{kg}$ **libre**. En la fila de arriba, $1\ \text{kg}$ vale $\$15$ (Libro). Total: $\$30 + \$15 = \mathbf{\$45}$.

Objeto \ Capacidad (w)	0 kg	1 kg	2 kg	3 kg	4 kg	5 kg
1. Libro (1 kg, $15)	0	15	15	15	15	15
2. Cámara (3 kg, $20)	0	15	15	20	35	35
3. Consola (4 kg, $30)	0	15	15	20	**35**	**45**

¿Qué objetos metimos? El Rastreo Inverso (*Backtracking*)

La celda $DP[3][5] = 45$ nos da la ganancia máxima, pero ¿cómo sabemos **qué objetos** la componen? Caminamos hacia atrás desde la esquina inferior derecha:

1. **En $DP[3][5] = 45$:** ¿Es igual al número de arriba ($DP[2][5] = 35$)? **NO** ($45 \neq 35$).
   - Significa que **SÍ metimos la Consola (4 kg)**. Restamos su peso: $5 - 4 = 1$ kg y subimos a $DP[2][1]$.
2. **En $DP[2][1] = 15$:** ¿Es igual al de arriba ($DP[1][1] = 15$)? **SÍ** ($15 = 15$).
   - Significa que **NO metimos la Cámara**. Subimos directo a $DP[1][1]$.
3. **En $DP[1][1] = 15$:** ¿Es igual al de arriba ($DP[0][1] = 0$)? **NO** ($15 \neq 0$).
   - **SÍ metimos el Libro (1 kg)**. Restamos $1 - 1 = 0$ kg. ¡Terminamos!

Variación Top-Down: Recursión con Memoización

Bottom-Up (Tabla Iterativa)

- Construye **todas** las celdas desde $0\,\text{kg}$ hasta W ($3 \times 5 = 15$ cálculos).
- Ideal cuando los pesos son pequeños y densos ($1, 2, 3 \ldots$).
- Cero riesgo de desbordar la pila (*Stack Overflow*).

Top-Down (Recursión + `memo`)

- Empieza preguntando por la meta `f(3, 5)` y **solo calcula los estados necesarios** (¡solo visita **6 celdas** de las 15!).
- Ideal cuando W es enorme ($W = 100{,}000$) y los pesos dan saltos grandes.

Tarea: *Implementación del algoritmo utilizando procesamiento Top-Down.*

Nota de Complejidad: $\mathcal{O}(n \cdot W)$ es **pseudo-polinomial**. Si W pasa de 5 a 10^{12}, la tabla explota en memoria aunque solo tengas 3 objetos.

¡Ahora les toca a ustedes!

Ejercicio en Clase (10 minutos)

Agreguemos un nuevo artículo tentador a nuestra mochila de $W = 5\,\text{kg}$:

- **Objeto 1 — Libro:** Peso = $1\,\text{kg}$ | Valor = $\$15$
- **Objeto 2 — Audífonos (¡Nuevo!):** Peso = $2\,\text{kg}$ | Valor = $\$18$
- **Objeto 3 — Cámara:** Peso = $3\,\text{kg}$ | Valor = $\$20$
- **Objeto 4 — Consola:** Peso = $4\,\text{kg}$ | Valor = $\$30$

Instrucciones:

1. Construyan en su cuaderno la matriz DP de 4×5.
2. Apliquen *Backtracking* para indicar exactamente qué objetos se llevan y si los nuevos Audífonos entran o no en la solución óptima.

Solución del Reto en el Aula

Objeto \ Capacidad (w)	0 kg	1 kg	2 kg	3 kg	4 kg	5 kg
0. Base	0	0	0	0	0	0
1. Libro (1 kg, $15)	0	15	15	15	15	15
2. Audífonos (2 kg, $18)	0	15	**18**	**33**	33	33
3. Cámara (3 kg, $20)	0	15	18	33	**35**	**38**
4. Consola (4 kg, $30)	0	15	18	33	35	**45**

- **Observación clave:** En la Fila 2 ($w = 3\ \text{kg}$), **Libro + Audífonos** suman $\$33$ (superando a la Cámara sola de $\$20$). En la Fila 3 ($w = 5\ \text{kg}$), **Audífonos + Cámara** alcanzan $\$38$.
- **Resultado Final ($w = 5\ \text{kg}$):** Al llegar a la Fila 4, **Consola ($4\ \text{kg}$) + Libro ($1\ \text{kg}$)** logran $\$45$, dejando fuera a los Audífonos.

2. La Mochila vista mediante Teoría de Grafos

¿Por qué vemos la Mochila en el bloque de Grafos?

A simple vista, la Mochila parece un problema de tablas numéricas sin grafos. Sin embargo, está en este módulo por **tres razones fundamentales**:

1. **Toda Programación Dinámica es un DAG:** Cada celda (i, w) es un **vértice** y cada decisión (llevar o no llevar) es una **arista dirigida**. Resolver la Mochila equivale a buscar el **Camino Más Largo (Mayor Valor)** en un Grafo Dirigido Acíclico.
2. **Grafos Implícitos (Árboles de Espacio de Estados):** A diferencia de Dijkstra (donde el mapa ya existe), aquí el grafo **se genera dinámicamente** conforme tomamos decisiones binarias (1 o 0).
3. **Recorridos de Grafos con Poda:** Técnicas como *Backtracking* (DFS) y *Branch & Bound* (BFS / Cola de Prioridad) son algoritmos puros de exploración de grafos.

La Mochila como Ruta Óptima en un DAG

Observa cómo nuestro Ejemplo 1 ($W = 5\,\text{kg}$) es en realidad un grafo por niveles desde el origen `(0 kg, $0)` hasta la meta óptima `(5 kg, $45)`:



Si desde el nodo `(3 kg, $20)` o `(4 kg, $35)` intentas tomar la arista **"+Consola (+4 kg)"**, generas nodos de $7\,\text{kg}$ y $8\,\text{kg}$ que son **podados por inviabilidad**.

Ramificación y Poda en la Mochila

¿Y si W tiene decimales o millones y no podemos crear una matriz DP?
Usamos un **Árbol de Ramificación y Poda** con una **Cota Superior (*Upper Bound* — UB)**.

El Superpoder: En cada nodo del árbol, calculamos cuánto ganaríamos en el mejor de los sueños rellenando el espacio libre de forma **Fraccionaria** (ordenando por ratio v_i/w_i).

Poda por Peso (Factibilidad)

Si al agregar un objeto superamos la capacidad ($w > W$), cortamos la rama de inmediato.

Poda por Cota ($UB \leq \text{Récord}$)

Si ni siquiera haciendo trampa (partiendo objetos) esa rama supera el mejor valor entero que ya tenemos seguro, **la podas completa**.

Árbol *Branch & Bound* sobre nuestro Reto ($W = 5\ \text{kg}$)

Ordenamos primero por ratio v_i/w_i: **1. Libro** ($\$15/\text{kg}$), **2. Audífonos** ($\$9/\text{kg}$), **3. Consola** ($\$7.5/\text{kg}$), **4. Cámara** ($\$6.67/\text{kg}$).

Nodo del Árbol	Decisión Tomada	Peso y Valor Real	Cota Superior (UB Fraccionaria)	Acción del Algoritmo
Nodo 0 (Raíz)	Inicio	$0\ \text{kg}, \$0$	$15 + 18 + \frac{2}{4}(30) = \mathbf{\$48}$	Expandir hacia Nivel 1
Nodo 1	SÍ Libro	$1\ \text{kg}, \$15$	$\$48.0$	**Expandir** (Mejor UB)
Nodo 2	NO Libro	$0\ \text{kg}, \$0$	$18 + \frac{3}{4}(30) = \mathbf{\$40.5}$	En espera $\rightarrow$ **¡PODADO!**
Nodo 3	SÍ Libro, SÍ Audífonos	$3\ \text{kg}, \$33$	$\$48.0$	Récord $= \$33$. Expandir
Nodo 4	SÍ Libro, NO Audífonos	$1\ \text{kg}, \$15$	$15 + 30\ (\text{Consola}) = \mathbf{\$45}$	En espera
Nodo 5	... + SÍ Consola (desde N3)	$7\ \text{kg}, \$63$	Inválido ($7 > 5\ \text{kg}$)	**¡PODADO POR PESO!**
Nodo 9	**SÍ Libro, NO Aud., SÍ Consola**	$5\ \text{kg}, \$45$	$\$45.0$	**¡ÓPTIMO! Récord** $= \$45$ *(Poda al Nodo 2 porque* $40.5 < 45$*)*

Implementación en Python: Mochila 0/1 + *Backtracking*

```python
def mochila_01(pesos, valores, nombres, capacidad_max):
    n = len(pesos)
    # Crear matriz DP de (n + 1) x (capacidad_max + 1) inicializada en 0
    dp = [[0 for _ in range(capacidad_max + 1)] for _ in range(n + 1)]

    # Llenar la tabla Bottom-Up fila por fila
    for i in range(1, n + 1):
        w_i, v_i = pesos[i - 1], valores[i - 1]
        for w in range(1, capacidad_max + 1):
            if w_i <= w:
                dp[i][w] = max(dp[i - 1][w], v_i + dp[i - 1][w - w_i])
            else:
                dp[i][w] = dp[i - 1][w]

    # Backtracking: reconstruir qué objetos fueron seleccionados
    elegidos, w_restante = [], capacidad_max
    for i in range(n, 0, -1):
        if dp[i][w_restante] != dp[i - 1][w_restante]:  # ¡El valor vino de incluir el objeto i!
            elegidos.append(nombres[i - 1])
            w_restante -= pesos[i - 1]

    return dp[n][capacidad_max], elegidos[::-1]

# Prueba: Ganancia óptima $45 -> ['Libro', 'Consola']
print(mochila_01([1, 2, 3, 4], [15, 18, 20, 30], ["Libro", "Audífonos", "Cámara", "Consola"], 5))
```

3. El Problema del Agente Viajero (TSP) y la Clase NP

La Pesadilla del Repartidor Global

Situación Problema

Dejamos lista la carga óptima del vehículo. Ahora el chofer debe salir del centro de distribución **A**, visitar a todos los clientes (**B, C, D...**) **exactamente una vez** y regresar a la base **A** al final del turno gastando el mínimo de combustible.

- **En teoría de grafos:** Encontrar el **Ciclo Hamiltoniano de Costo Mínimo**.
- **¿Por qué no usamos Dijkstra o Floyd-Warshall?** Porque Dijkstra busca ir de A a B usando la menor cantidad de nodos intermedios posibles, ¡mientras que aquí estamos **obligados a visitar TODOS los vértices sin repetir**!

La Explosión Combinatoria: ¿Por qué falla la Fuerza Bruta?

En un grafo simétrico de n ciudades, si fijamos la ciudad de inicio, el número de rutas posibles es:

$$\text{Rutas} = \frac{(n-1)!}{2}$$

Ciudades (n)	Rutas Posibles	Tiempo en Supercomputadora (10^9 rutas/seg)
5 ciudades	12 rutas	Instantáneo (0.00000001 seg)
10 ciudades	$181{,}440$ rutas	0.00018 segundos
15 ciudades	$43{,}589{,}145{,}600$ rutas	43.5 segundos
21 ciudades	1.21×10^{18} rutas	¡38.5 AÑOS!

¿Qué significa que un problema sea NP, NP-Completo o NP-Hard?

Clase P (Polinomial)

Problemas **fáciles de resolver** en tiempo $\mathcal{O}(n^k)$.

Ejemplos: Ordenamiento, Dijkstra, Floyd-Warshall, Kruskal, Mochila Fraccionaria.

Clase NP (Polinomial No Determinista)

Problemas donde **verificar** una solución es rapidísimo, aunque encontrarla sea difícil.

Ejemplo: "¿Existe una ruta menor a 80 km?". Si te dan la ruta, sumas en 1 seg y verificas.

NP-Completo

Los problemas más difíciles dentro de NP. Si descubres cómo resolver **uno solo** en tiempo polinomial, ¡resuelves todos los demás automáticamente!

NP-Hard (NP-Difícil)

Problemas de optimización (como hallar **el mínimo exacto del TSP**) donde ni siquiera es fácil verificar si ya tienes el óptimo absoluto sin evaluar el resto.

El Mapa de Estrategias frente al TSP

Como un ingeniero no puede esperar 38 años a que termine de correr su programa, atacamos el TSP con **tres familias de algoritmos** según el tamaño de n:

1. Exacto con Poda

Usa **Cotas Inferiores (LB)** para descartar millones de ramas malas sin verlas.

Ideal: $n \leq 40$

➔

2. Programación Dinámica

Guarda subconjuntos visitados con máscaras de bits: $\mathcal{O}(n^2 \cdot 2^n)$.

Ideal: $n \leq 22$ *(*0.4 *seg)*

➔

3. Heurísticas y Aproximación

Vecino Más Cercano + **2-Opt** o **Christofides** ($1.5\times$ óptimo).

Ideal: $n > 1{,}000$ *ciudades*

Acto 1: La Trampa de la Heurística *Greedy* (Vecino Más Cercano)

¿Qué pasa si siempre vamos a la ciudad más cercana disponible?

- Al principio ahorras muchos kilómetros con decisiones localmente baratas.
- Pero al final te quedas atrapado lejos del origen y tienes que cruzar todo el mapa para regresar, **formando cruces en "X"** en la ruta.

El Antídoto Industrial (Mejora Local 2-Opt):
Si dos aristas de tu ruta se cruzan en forma de **X**, bórralas y reconéctalas en paralelo. Por la *desigualdad triangular*, **descruzar dos caminos siempre reduce la distancia total** en tiempo $\mathcal{O}(n^2)$.

Acto 2: Solución Exacta en el Pizarrón (*Branch & Bound*)

El Cambio de Chip respecto a la Mochila:

- En la Mochila **maximizábamos** ($\$$) $\rightarrow$ usábamos **Cota Superior (UB)**.
- En el TSP **minimizamos** (km) $\rightarrow$ usamos **Cota Inferior (*Lower Bound* — LB)**.
- **Regla de Poda:** Si la promesa mínima de una rama ya es mayor o igual a una ruta completa conocida ($LB \geq \mathrm{Mejor_Costo}$), **¡PODAMOS LA RAMA!**

¿Cómo calculamos el LB rápido en el pizarrón?
Para completar el ciclo, estamos obligados a **salir exactamente una vez de cada ciudad**. Sumamos el costo de las carreteras ya elegidas más **la salida válida más barata disponible** de cada ciudad restante.

Ejemplo TSP Paso a Paso: Las 4 Ciudades (A, B, C, D)

Buscamos el ciclo mínimo saliendo y regresando a **A**:

Desde \ Hacia	A	B	C	D	Salida más barata de cada fila
A	∞	10	15	20	10 km (hacia B)
B	10	∞	35	25	10 km (hacia A)
C	15	35	∞	30	15 km (hacia A)
D	20	25	30	∞	20 km (hacia A)

Cota Inferior en la Raíz (Nodo 0 — Inicio en A):

$$LB(\text{Raíz}) = \underbrace{10}_{\text{A}} + \underbrace{10}_{\text{B}} + \underbrace{15}_{\text{C}} + \underbrace{20}_{\text{D}} = \mathbf{55}\ \text{km}$$

(Ningún tour posible en este mapa puede medir menos de $55\ \text{km}$*).*

TSP Paso 1: Nivel 1 del Árbol (Saliendo de A)

Regla de oro: Si haces $A \rightarrow B$, desde **B está prohibido volver directo a A** (formarías un subciclo $A \rightarrow B \rightarrow A$ dejando fuera a C y D).

Nodo 1: Ruta A ➔ B

- Salida de A (fija a B): **10**
- Salida de B (sin A, entre C=35 y D=25): **25**
- Salida de C (15) + Salida de D (20)

LB = 10 + 25 + 15 + 20 = 70 km

Nodo 2: Ruta A ➔ C

- Salida de A (fija a C): **15**
- Salida de B (10) + Salida de D (20)
- Salida de C (sin A, entre B=35 y D=30): **30**

LB = 15 + 10 + 30 + 20 = 75 km

Nodo 3: Ruta A ➔ D (LB = 20 + 10 + 15 + 25 = 70 km)

Como la matriz es **simétrica**, empezar o terminar por D es el espejo exacto de las rutas por B y C.
¡Elegimos expandir primero el **Nodo 1 (A ➔ B con LB = 70 km)**!

TSP Paso 2: Hallar el Óptimo (80 km) y Podar el Árbol

Expandiendo el Nodo 1 ($A \to B$, $LB = 70$ km):

- **Nodo 4 (Ruta $A \to B \to D \to C \to A$):**
  - Costo real $= 10\ (\mathrm{A} \to \mathrm{B}) + 25\ (\mathrm{B} \to \mathrm{D}) + 30\ (\mathrm{D} \to \mathrm{C}) + 15\ (\mathrm{C} \to \mathrm{A}) = \mathbf{80}$ km.
  - **¡Actualizamos nuestro Récord Global:** `Mejor_Costo = 80 km` **!**
- **Nodo 5 (Ruta $A \to B \to C \ldots$):**
  - $LB = 10\ (\mathrm{A} \to \mathrm{B}) + 35\ (\mathrm{B} \to \mathrm{C}) + 30\ (\mathrm{C} \to \mathrm{D}) + 20\ (\mathrm{D} \to \mathrm{A}) = \mathbf{95}$ km.
  - Como $95 \geq 80$ km $\to$ **¡PODADO POR COTA INFERIOR!**

Poda en cascada del resto del árbol:

- Al revisar el **Nodo 2 ($A \to C$)**, su hijo $A \to C \to B$ da $LB = 95$ km ≥ 80 (**¡PODADO!**) y $A \to C \to D$ da 80 km (ruta espejo inversa).
- **Solución Óptima Garantizada:** $A \to B \to D \to C \to A$ **(**80 km**)**.

El Enfoque Alternativo: Programación Dinámica de Held-Karp

¿Cómo se aplica Programación Dinámica al TSP?
En un mapa de 10 ciudades, recorrer $A \to B \to C \to D$ o recorrer $A \to C \to B \to D$ deja exactamente el **mismo subproblema hacia adelante**:

- *"Estoy parado en la ciudad D y ya visité el subconjunto $S = \{A, B, C, D\}$".*

Ecuación de Held-Karp (Con Máscara de Bits `bitmask`):

$$g(S, i) = \min_{k \in S,\, k \neq i} \Big\{ g(S \setminus \{i\}, k) + d(k, i) \Big\}$$

- Reduce la complejidad de $\mathcal{O}(n!)$ a $\mathcal{O}(n^2 \cdot 2^n)$.
- Para 20 ciudades, pasa de **38 años** a **0.4 segundos**.

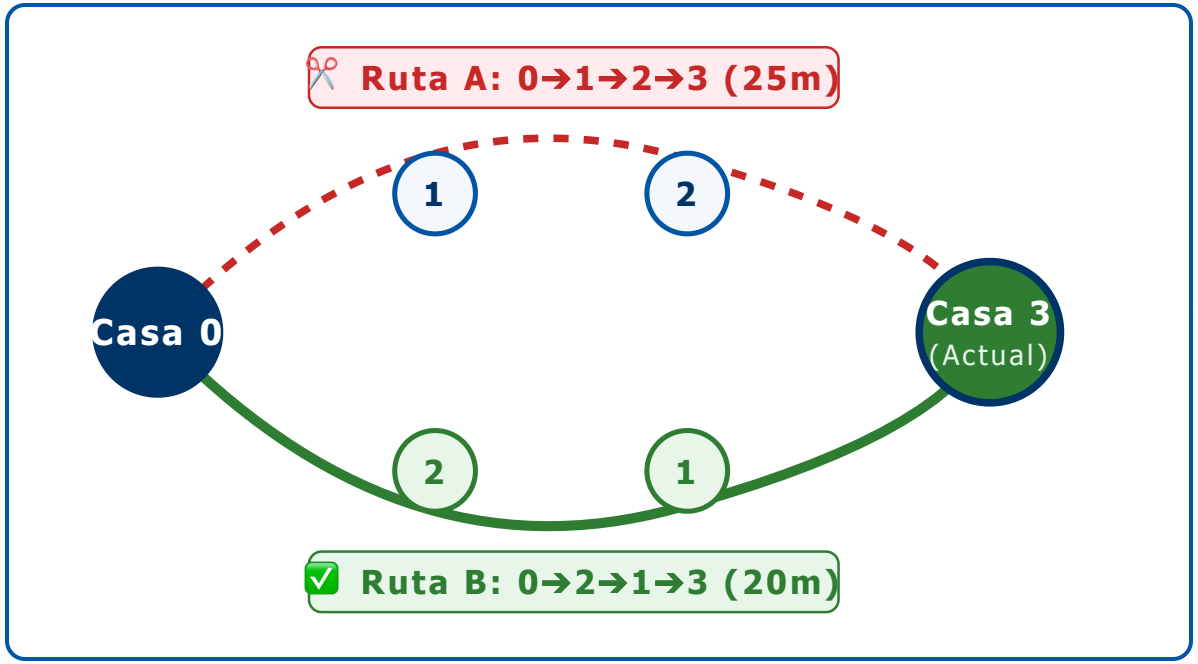
Acto 3: Programación Dinámica de Held-Karp

La Metáfora del Repartidor de Helados
Imagina que sales de tu **Casa (Ciudad 0)** con un carrito de helados y debes entregar pedidos en las casas de **3 amigos (Ciudades 1, 2 y 3)** y volver a casa antes de que se derritan.

Sin Libreta (Fuerza Bruta):
Pruebas todas las combinaciones a lo loco. Con 20 casas son $19! \approx 1.21 \times 10^{17}$ caminos (**~3.8 años** de cómputo).

Con Libreta Mágica (Held-Karp):
Anotas los mejores caminos parciales para **jamás calcular dos veces lo mismo**. Con 20 casas toma **~0.4 segundos**.

El Gran Secreto de Held-Karp: El Olvido Inteligente



¿Qué tienen en común A y B?

Ambas te dejan parado en la **misma Casa 3** habiendo tachado exactamente el **mismo grupo** `{0, 1, 2, 3}`.

¡La Regla de Oro!

Para tu futuro (ir a la casa 4, 5...), **¡NO IMPORTA en qué orden visitaste 1 y 2!** Solo importa el grupo visitado (`S`) y dónde estás hoy (`i`).
Tiramos la Ruta A para siempre.

La Libreta Mágica: ¿Cómo se lee la Fórmula?

Cada página de nuestra libreta se llama $dp(S, i)$ y guarda el costo más barato con solo dos datos:

- S **(La Bolsita):** El grupo de casas que ya visitaste (sin importar el orden).
- i **(Tu Ubicación):** La última casa donde estás parado ahorita mismo ($i \in S$).

Ecuación de Recurrencia de Bellman-Held-Karp:

$$dp(S, i) = \min_{k \in S,\, k \neq i} \left\{ \underbrace{dp(S \setminus \{i\}, k)}_{\text{Mejor ruta al grupo sin } i \text{ terminando en } k} + \underbrace{\text{dist}(k \to i)}_{\text{Último paso de } k \text{ a } i} \right\}$$

🏠 **NIVEL 1**

Bolsitas de 1

Ir directo de Casa 0 a cada amigo.

➔

🤝 **NIVEL 2**

Bolsitas de 2

Usar el Nivel 1 para armar parejas óptimas.

➔

✂️ **NIVEL 3**

Bolsitas de 3

Comparar candidatos y **podar los caros**.

➔

🏆 **META FINAL**

Cerrar el Ciclo

Dar el último paso de regreso a **Casa 0**.

La Libreta Mágica: La Fórmula en Acción

La Ecuación de Recurrencia de Bellman-Held-Karp:

$$dp(S, i) = \min_{k \in S,\, k \neq i} \left\{ \underbrace{dp(S \setminus \{i\}, k)}_{\text{Mejor ruta previa al grupo sin } i \text{ terminando en } k} + \underbrace{\text{dist}(k \to i)}_{\text{Costo del paso de } k \text{ a } i} \right\}$$

Donde `S` es la bolsita de casas ya visitadas e `i` es la ciudad donde estás parado ahorita mismo.

🏠 **NIVEL 1**

Bolsitas de 1

Caso base: directo de Casa 0

```
dp({1}, 1) = dist(0 → 1)
= 10
```

S={1}, i=1 (sin escalas)

➔

🤝 **NIVEL 2**

Bolsitas de 2

Un solo amigo previo (k=1)

```
dp({1,2}, 2) =
dp({1}, 1) + dist(1 → 2)
10 + 35 = 45
```

S={1,2}, i=2, k=1

➔

✂️ **NIVEL 3**

Bolsitas de 3

¡Duelo de opciones y Poda!

```
dp({1,2,3}, 2) = min(
 dp({1,3},1)+d(1→2) [80],
 dp({1,3},3)+d(3→2) [65]
) = 65
```

S={1,2,3}, i=2, k∈{1,3}

➔

🏆 **META FINAL**

Cerrar Ciclo

Último salto a Casa 0

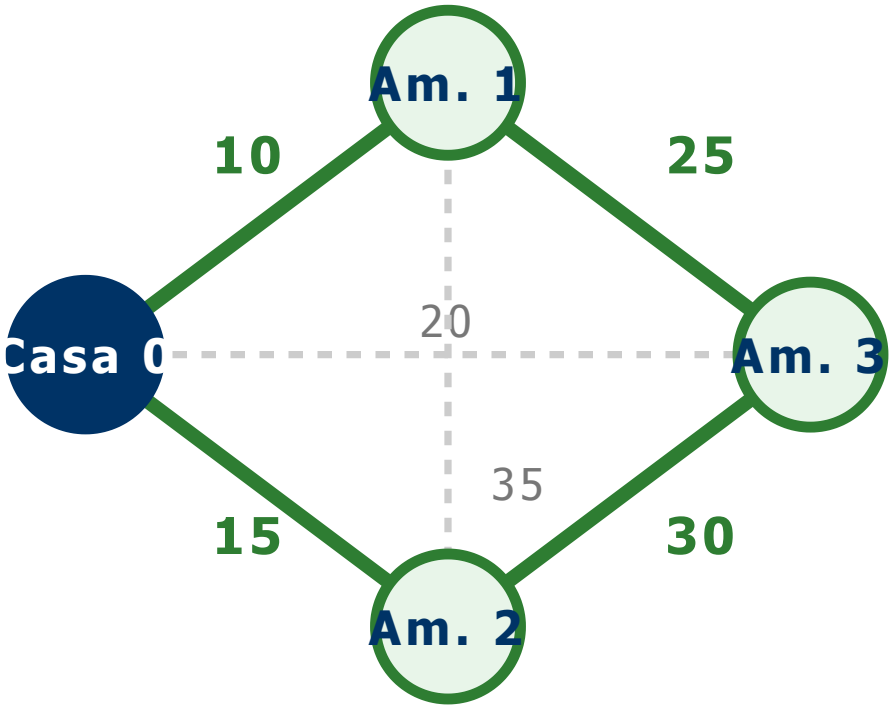
```
min {
 dp(Todos, 1) + d(1→0) [80],
 dp(Todos, 2) + d(2→0) [80],
 dp(Todos, 3) + d(3→0) [95]
} = 80
```

Salida: 0➔1➔3➔2➔0

Ejemplo Guiado: El Mapa y Niveles 1 y 2

Desde \ Hacia	0	1	2	3
Casa 0	0	10	15	20
Amigo 1	10	0	35	25
Amigo 2	15	35	0	30
Amigo 3	20	25	30	0

Nivel 1 (Salir de Casa 0):
$dp(\{1\}, 1) = \mathbf{10}$ | $dp(\{2\}, 2) = \mathbf{15}$ | $dp(\{3\}, 3) = \mathbf{20}$



Nivel 2: Bolsitas de 2 amigos (Usando el Nivel 1 sin recalcular):

- $\{1, 2\}$: En **2** (desde 1): $10 + 35 = \mathbf{45}$ | En **1** (desde 2): $15 + 35 = \mathbf{50}$
- $\{1, 3\}$: En **3** (desde 1): $10 + 25 = \mathbf{35}$ | En **1** (desde 3): $20 + 25 = \mathbf{45}$
- $\{2, 3\}$: En **3** (desde 2): $15 + 30 = \mathbf{45}$ | En **2** (desde 3): $20 + 30 = \mathbf{50}$

Ejemplo Guiado: Nivel 3 (¡La Poda!) y Regreso a Casa

Nivel 3 — Bolsita $\{1, 2, 3\}$ (Dos candidatos para cada meta, ¡solo sobrevive el más barato!):

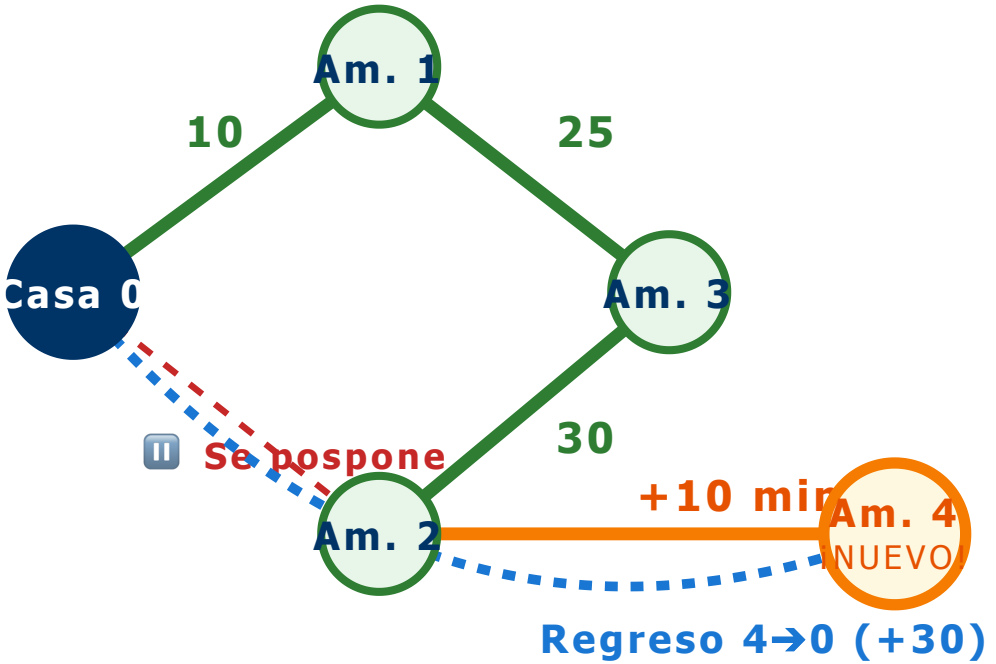
- **Terminando en 3** ($dp(\{1, 2, 3\}, 3)$): ¿Vengo de 1 ($50 + 25 = 75$) o de 2 ($45 + 30 = 75$)? $\rightarrow$ Gana 75.
- **Terminando en 2** ($dp(\{1, 2, 3\}, 2)$): ¿Vengo de 1 ($45 + 35 = 80$) o de 3 ($35 + 30 = \mathbf{65}$)? $\rightarrow$ Gana 65 *(¡Poda el 80!)*.
- **Terminando en 1** ($dp(\{1, 2, 3\}, 1)$): ¿Vengo de 2 ($50 + 35 = 85$) o de 3 ($45 + 25 = \mathbf{70}$)? $\rightarrow$ Gana 70 *(¡Poda el 85!)*.

Nivel Final — Cerrar el ciclo regresando a Casa 0:

- Desde **1**: $70 + \text{dist}(1 \rightarrow 0) = 70 + 10 = \mathbf{80}$
- Desde **2**: $65 + \text{dist}(2 \rightarrow 0) = 65 + 15 = \mathbf{80}$
- Desde **3**: $75 + \text{dist}(3 \rightarrow 0) = 75 + 20 = 95$
- **¡Costo Mínimo Óptimo =** 80**!** (Ruta ganadora: $0 \xrightarrow{+10} 1 \xrightarrow{+25} 3 \xrightarrow{+30} 2 \xrightarrow{+15} 0$).

¡Sorpresa! Se muda al barrio el Amigo 4

Nuevo Escenario: Justo cuando ibas a salir, el **Amigo 4** pide un helado. Vive a 10 min **del Amigo 2**, a 30 min **de tu Casa 0**, a 40 min **del Amigo 1** y a 50 min **del Amigo 3**. ¿Se arruina nuestra libreta?



Lo que se SALVA al 100%:
¡Todas las páginas de tu libreta para `{1, 2, 3}` ($dp = 70, 65, 75$) **no se recalculan**! Siguen siendo verdades absolutas.

Lo único que CAMBIA:
Cancelamos el regreso prematuro a Casa 0 (el 80) y creamos un **Nivel 4** usando los récords del Nivel 3 como escalón.

¿Cómo se Recicla el Nivel 3 para llegar al Amigo 4?

Queremos saber cómo visitar a los 4 amigos y **terminar parado en el Amigo 4**: $dp(\{1,2,3,4\},4)$. ¡Mira cómo la libreta nos ahorra la mitad del trabajo!

Con Fuerza Bruta (Empezar de cero):
Tendrías que probar las **6 rutas** desde Casa 0:

1. $0 \to 2 \to 3 \to 1 \to 4$ *(85 + 40 = 125)*
2. $0 \to 3 \to 2 \to 1 \to 4$ *(70 + 40 = 110)*
3. $0 \to 3 \to 1 \to 2 \to 4$ *(80 + 10 = 90)*
4. $0 \to 1 \to 3 \to 2 \to 4$ *(65 + 10 = 75)*
5. $0 \to 1 \to 2 \to 3 \to 4$ *(75 + 50 = 125)*
6. $0 \to 2 \to 1 \to 3 \to 4$ *(75 + 50 = 125)*

Con Held-Karp (Solo 3 sumas):
Los caminos perdedores (**80** y **85**) **ya estaban muertos**. Solo miramos los 3 ganadores del Nivel 3:

- Desde **1**: $\mathbf{70} + \mathrm{d}(1 \to 4) = 70 + 40 = 110$
- Desde **2**: $\mathbf{65} + \mathrm{d}(2 \to 4) = 65 + 10 = \mathbf{75}$
- Desde **3**: $\mathbf{75} + \mathrm{d}(3 \to 4) = 75 + 50 = 125$

¡Gana 75**!** Y al volver a Casa 0 $(+30)$:
Nuevo Óptimo:
$0 \to 1 \to 3 \to 2 \to 4 \to 0 = \mathbf{105}$ min.

Entonces ¿Cómo enfrentamos al Amigo 4 con DP?

El Error de Novato (La Trampa Voraz):
Pensar: *"Mi ruta ganadora era* $0 \to 1 \to 3 \to 2 \to 0$*. ¡Ah! Como el Amigo 4 vive cerca del 2, simplemente lo meto ahí:* $0 \to 1 \to 3 \to 2 \to \mathbf{4} \to 0$ *y listo"*.
¡FALSO! Si haces eso, podrías perderte un atajo gigantesco. Quizás el Amigo 4 era el puente perfecto para pasar del 1 al 2.

La Estrategia Held-Karp (La Expansión de la Libreta):
No parchamos la ruta, **expandimos la libreta**.

1. **Conservamos** las 16 páginas viejas donde el Amigo 4 NO está (`0xxxx`).
2. **Agregamos** 16 páginas nuevas donde el Amigo 4 SÍ está (`1xxxx`).
3. Llenamos las páginas nuevas **usando los resultados de las viejas**.

Construyendo las Páginas Nuevas (El Amigo 4 se infiltra)

Para saber si el Amigo 4 debe ir al principio, en medio o al final, el algoritmo arma bolsitas combinándolo con las casas que **ya teníamos resueltas**.

Paso 1: Amigo 4 como "Atajo"

Bolsitas de 2 y 3

Mezclamos al 4 con los demás.

Para calcular `{1, 4}`:
Usamos el viejo `{1}` + viaje al 4.
Para calcular `{1, 2, 4}`:
Usamos `{1, 4}` o `{2, 4}`.

➔

Paso 2: El Nivel 4

Todos antes de volver

¿Quién debe ser el último?

Calculamos `dp({1,2,3,4}, X)` probando terminar en X=1, 2, 3 o 4.
(¡Aquí es donde se descubre si el 4 debió ir en medio o al final!)

➔

Paso 3: El Gran Duelo

Regreso a Casa 0

La competencia final.

Sumamos el viaje a Casa 0 desde los 4 posibles finales.
¡El ganador nos dará la ruta absoluta y perfecta!

El Duelo Final: ¿Dónde quedó el Amigo 4?

¿Y si el 4 quedó al final?

Terminaríamos en el Amigo 4: `dp({1,2,3,4}, 4)` + regreso a casa.

Esto usaría las páginas viejas del Nivel 3 (terminar en 1, 2 o 3 y saltar al 4).

¿Y si el 4 quedó en medio?

Terminaríamos, por ejemplo, en el Amigo 2: `dp({1,2,3,4}, 2)` + regreso a casa.

Esto significa que la ruta pasó por el 4 antes de llegar al 2 (ej. 0➔1➔4➔3➔2➔0).

La Belleza del Algoritmo

Como el algoritmo calcula **TODAS** las páginas nuevas (terminando en 1, 2, 3 y 4) y las enfrenta en un duelo final (`min`), **es matemáticamente imposible que se le escape un atajo**. ¡Siempre encontrará si el Amigo 4 era mejor al principio, en medio o al final!

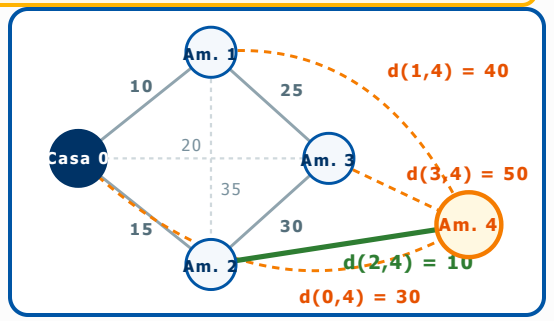
¡Ahora les toca a ustedes! El Reto del Amigo 4

Ejercicio en Clase (10 min): El **Amigo 4** entra al mapa. Usando su libreta anterior (sin recalcular lo que ya sabemos de `{1, 2, 3}`), encuentren a mano la **nueva ruta óptima** y su costo total.

Desde \ Hacia	0	1	2	3	🌟 4
Casa 0	0	10	15	20	30
Amigo 1	10	0	35	25	40
Amigo 2	15	35	0	30	10
Amigo 3	20	25	30	0	50
🌟 **Amigo 4**	30	40	10	50	0

Páginas viejas listas para reutilizar:

- **Nivel 1:** $dp(\{1\},1) = 10$ | $dp(\{2\},2) = 15$ | $dp(\{3\},3) = 20$
- **Nivel 2:** $dp(\{1,2\},2) = 45, dp(\{1,2\},1) = 50$ | $dp(\{1,3\},3) = 35, dp(\{1,3\},1) = 45$ | $dp(\{2,3\},3) = 45, dp(\{2,3\},2) = 50$
- **Nivel 3:** $dp(\{1,2,3\},1) = \mathbf{70}$ | $dp(\{1,2,3\},2) = \mathbf{65}$ | $dp(\{1,2,3\},3) = \mathbf{75}$



Pasos a resolver en su cuaderno:

1. Calculen las páginas nuevas con el **4**: Nivel 1 ($\{4\}$), Nivel 2 ($\{1,4\}, \{2,4\}, \{3,4\}$) y Nivel 3 ($\{1,2,4\}, \{1,3,4\}, \{2,3,4\}$).
2. Comparen los 4 finales en el **Nivel 4** ($\{1,2,3,4\}$) + regreso a **Casa 0**.

Solución del Reto en el Aula: La Libreta del Amigo 4

1. Nuevas páginas con el Amigo 4 (Niveles 1 al 3):

- **Nivel 1:** $dp(\{4\}, 4) = \mathbf{30}$
- **Nivel 2 (Parejas con el 4):**
  - $\{1,4\}$: en $4 \to 10 + 40 = \mathbf{50}$ | en $1 \to 30 + 40 = \mathbf{70}$
  - $\{2,4\}$: en $4 \to 15 + 10 = \mathbf{25}$ | en $2 \to 30 + 10 = \mathbf{40}$
  - $\{3,4\}$: en $4 \to 20 + 50 = \mathbf{70}$ | en $3 \to 30 + 50 = \mathbf{80}$
- **Nivel 3 (El 4 infiltrado en tríos):**
  - $\{1,2,4\}$: en $1 = \mathbf{65}$, en $2 = \mathbf{60}$, en $4 = \mathbf{55}$
  - $\{1,3,4\}$: en $1 = \mathbf{105}$, en $3 = \mathbf{95}$, en $4 = \mathbf{85}$
  - $\{2,3,4\}$: en $2 = \mathbf{80}$, en $3 = \mathbf{70}$, en $4 = \mathbf{55}$

2. Nivel 4 ($\{1,2,3,4\}$) y Regreso a Casa 0:

- **Terminando en 4** *(usa el viejo $\{1,2,3\}$)*:
  $\min(70 + 40,\ \mathbf{65 + 10},\ 75 + 50) = \mathbf{75}$
  $\to$ Regreso a Casa 0 ($+30$): $75 + 30 = \mathbf{105}$ min 🏆
- **Terminando en 1** *(usa $\{2,3,4\}$)*:
  $\min(80 + 35,\ \mathbf{70 + 25},\ 55 + 40) = \mathbf{95}$
  $\to$ Regreso a Casa 0 ($+10$): $95 + 10 = \mathbf{105}$ min 🏆
- **Terminando en 2** *(usa $\{1,3,4\}$)*:
  $\min(105 + 35,\ 95 + 30,\ \mathbf{85 + 10}) = \mathbf{95}$
  $\to$ Regreso a Casa 0 ($+15$): $95 + 15 = 110$ min ❌
- **Terminando en 3** *(usa $\{1,2,4\}$)*:
  $\min(65 + 25,\ \mathbf{60 + 30},\ 55 + 50) = \mathbf{90}$
  $\to$ Regreso a Casa 0 ($+20$): $90 + 20 = 110$ min ❌

Resultado Óptimo: 105 min — Ruta: $0 \xrightarrow{+10} 1 \xrightarrow{+25} 3 \xrightarrow{+30} 2 \xrightarrow{+10} 4 \xrightarrow{+30} 0$ *(o su espejo inverso terminando en 1:* $0 \to 4 \to 2 \to 3 \to 1 \to 0$*).*

¿Por qué se le llama DP con *Bitmask* (Máscara de Bits)?

La Metáfora del Tablero de Interruptores: En lugar de usar listas lentas como `{Casa 0, Amigo 1, Amigo 3}`, representamos cada ciudad con un interruptor (`1` = **Visitada**, `0` = **No visitada**). ¡Así cada subconjunto es un simple **número entero**!

Amigo 3 ($2^3 = 8$)	Amigo 2 ($2^2 = 4$)	Amigo 1 ($2^1 = 2$)	Casa 0 ($2^0 = 1$)
1	**0**	**1**	**1**
VISITADO (+8)	PENDIENTE (+0)	VISITADO (+2)	VISITADO (+1)

Bolsita `{0, 1, 3}` ➔ Binario `1011` ➔ Suma: $8 + 0 + 2 + 1 = 11$ ➔ ¡Vamos directo al cajón `dp[11][u]`!

Los 5 Hechizos de Bits (Operadores *Bitwise* en $O(1)$)

El procesador evalúa y modifica subconjuntos enteros en **un solo ciclo de reloj** usando estos 5 operadores:

Expresión en Código	Nombre del Hechizo	¿Qué hace con los interruptores? (Ejemplo con $n = 4$)
`1 << n`	Total de Bolsitas (2^n)	Empuja el `1` cuatro lugares a la izquierda: `10000` $= \mathbf{16}$ filas.
`1 << u`	Dedo Señalador	Crea una máscara con **solo el interruptor `u` encendido** (ej. $u = 2 \rightarrow$ `0100`).
`mask & (1 << u)`	Mirar por el Agujero (`AND`)	Pregunta: **¿Ya visité `u`?** Da $\neq 0$ si ambos tienen `1` en la posición `u`.
`mask | (1 << v)`	Encender Interruptor (`OR`)	**Viajar a `v`:** Une los agujeros. Ej: `1011
`mask ^ (1 << u)`	Apagar Interruptor (`XOR`)	**Quitar ciudad `u`:** Invierte ese bit. Ej: `1111 ^ 0100` $\rightarrow$ **`1011`** (11).

La Meta Final `(1 << n) - 1`: Si `1 << 4` es `10000` (16), al restarle 1 obtenemos **`01111`** **(15)** $\rightarrow$ ¡Todos los interruptores encendidos!

El Truco de las Migajas de Pan: Reconstruyendo la Ruta

Saber que el costo mínimo es 80 no basta; el chofer necesita el **orden exacto de las calles**. Para no olvidar el camino, creamos una segunda libreta `parent[mask][v]` donde anotamos **quién nos trajo hasta ahí**.

1. En Amigo 2 (1111)

`parent[15][2]`

dice:

"Viniste del 3"

.

Apagamos bit 2 con

`^`

➔

2. En Amigo 3 (1011)

`parent[11][3]`

dice:

"Viniste del 1"

.

Apagamos bit 3 con

`^`

➔

3. En Amigo 1 (0011)

`parent[3][1]`

dice:

"Viniste de 0"

.

Apagamos bit 1 con

`^`

➔

4. Invertir Lista

Leímos

`[0, 2, 3, 1, 0]`

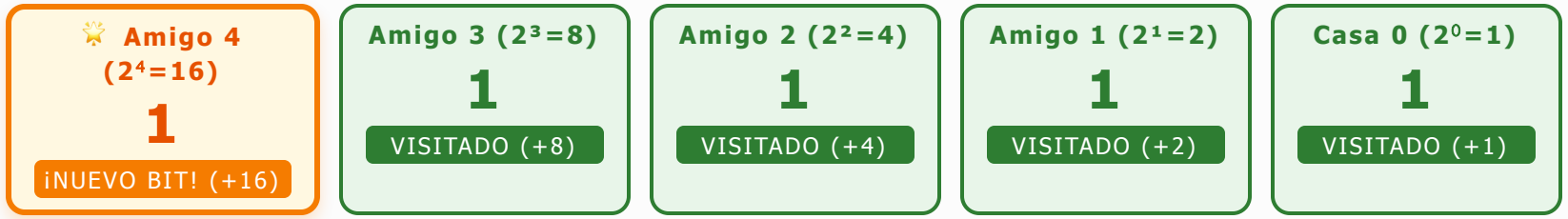
.

Al invertir:

0 ➔ 1 ➔ 3 ➔ 2 ➔ 0

El Efecto del Amigo 4 en los Interruptores (*Bitmask*)

Agregar al **Amigo 4** simplemente instala **un 5.º interruptor a la izquierda ($2^4 = 16$)**. Toda tu tabla anterior son las filas con ese interruptor apagado (`0xxxx`), y las nuevas son con él encendido (`1xxxx`):



Métrica	Antes (Casa 0 + 3 Amigos)	Ahora (Casa 0 + 4 Amigos)	¿Qué pasó al crecer n?
Bits de la Máscara	`4 bits` (`0000` a `1111`)	`5 bits` (`00000` a `11111`)	Se agrega **1 interruptor** a la izquierda.
Filas en la Libreta (2^n)	`1 << 4` = 16 **filas**	`1 << 5` = 32 **filas**	La libreta solo se **duplica** ($\times 2$), no explota en $\times n!$.
Meta `todos_visitados`	15 (`1111`)	31 (`11111`)	$15 + 16\ (\text{nuevo bit}) = \mathbf{31}$.

Implementación en Python: Held-Karp Iterativo (*Bottom-Up*)

```python
def held_karp_bottom_up(dist):
    n, INF = len(dist), float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
    dp[1][0] = 0  # Caso base: Casa 0 encendida (mask = 0001 = 1)

    for mask in range(1, 1 << n):
        if not (mask & 1): continue  # Casa 0 siempre debe estar incluida
        for u in range(n):
            if not (mask & (1 << u)) or dp[mask][u] == INF: continue
            for v in range(n):
                if mask & (1 << v): continue  # Si ya visitamos 'v', saltar
                next_mask = mask | (1 << v)   # Encendemos interruptor de 'v'
                if dp[mask][u] + dist[u][v] < dp[next_mask][v]:
                    dp[next_mask][v] = dp[mask][u] + dist[u][v]
                    parent[next_mask][v] = u  # Migaja


    todos = (1 << n) - 1
    mejor_costo, curr_city = min((dp[todos][u] + dist[u][0], u) for u in range(1, n))
    camino, curr_mask = [0], todos
    while curr_city != 0:
        camino.append(curr_city)
        curr_mask, curr_city = curr_mask ^ (1 << curr_city), parent[curr_mask][curr_city]
    return mejor_costo, (camino + [0])[::-1]  # Costo: 80, Ruta: [0, 1, 3, 2, 0]
```

Implementación en Python: Held-Karp Recursivo (*Top-Down*)

```python
def held_karp_top_down(dist):
    n, INF = len(dist), float('inf')
    todos = (1 << n) - 1
    memo = [[-1] * n for _ in range(1 << n)]
    next_city = [[-1] * n for _ in range(1 << n)]

    def resolver(mask, u):
        if mask == todos: return dist[u][0]        # Caso Base: volver a Casa 0
        if memo[mask][u] != -1: return memo[mask][u]  #Ya está
        mejor_costo = INF
        for v in range(n):
            if not (mask & (1 << v)):              # Si el interruptor 'v' está apagado
                costo = dist[u][v] + resolver(mask | (1 << v), v)
                if costo < mejor_costo:
                    mejor_costo, next_city[mask][u] = costo, v
        memo[mask][u] = mejor_costo
        return mejor_costo

    costo_minimo = resolver(1, 0)  # Iniciamos en Casa 0 con máscara 0001 (1)
    camino, curr_mask, curr_u = [0], 1, 0
    while curr_mask != todos:
        sig = next_city[curr_mask][curr_u]
        camino.append(sig)
        curr_mask, curr_u = curr_mask | (1 << sig), sig
    return costo_minimo, camino + [0]  # Costo: 80, Ruta: [0, 1, 3, 2, 0]
```

Duelo de Enfoques en Held-Karp: *Bottom-Up* vs. *Top-Down*

Ambos algoritmos tienen exactamente la misma complejidad asintótica de tiempo $\mathcal{O}(n^2 \cdot 2^n)$ y espacio $\mathcal{O}(n \cdot 2^n)$, pero se comportan distinto al programarlos:

Característica	*Bottom-Up* (Iterativo con `for`)	*Top-Down* (Recursivo con `memo`)
Forma de pensar	**Desde el pasado:** *"¿De qué ciudad u me convenía venir para llegar a v?"*	**Hacia el futuro:** *"¿A qué ciudad v me conviene ir ahora desde u?"*
Estados explorados	Recorre todas las filas `1 << n` (filtrando con `mask & 1`).	**Solo visita estados alcanzables** con el bit 0 encendido automáticamente.
Reconstrucción de Ruta	**Hacia atrás** usando `parent` y apagando bits con `^` (**XOR**). Requiere `reverse()`.	**Hacia adelante** usando `next_city` y encendiendo bits con `|` (**OR**). Directo.
Rendimiento en CPU	**Más rápido en la práctica** (bucles planos sin sobrecarga de llamadas recursivas).	Más intuitivo de escribir en pizarra o entrevistas en menos líneas.

Resumen Comparativo: Mochila 0/1 vs. TSP

Característica	Mochila 0/1 (*Knapsack*)	Agente Viajero (*TSP*)
Objetivo	**Maximizar** valor sin superar peso W	**Minimizar** costo visitando todos los nodos
Fuerza Bruta	$\mathcal{O}(2^n)$ subconjuntos	$\mathcal{O}(n!)$ permutaciones
Programación Dinámica	Matriz $DP[i][w]$ en $\mathcal{O}(n \cdot W)$	Held-Karp $DP[\text{mask}][i]$ en $\mathcal{O}(n^2 \cdot 2^n)$
Branch & Bound	Usa **Cota Superior (UB)** fraccionaria	Usa **Cota Inferior (LB)** de salidas mínimas
Visión en Grafos	Camino más largo en un **DAG por niveles**	Ciclo Hamiltoniano mínimo en **Grafo Completo**

4. Asignación de Estudio Independiente

Reto de Ingeniería: Selección y Justificación de Arquitectura TSP

Caso de Estudio (Entrega Individual / Equipos)

Una startup de logística en México te contrata para diseñar el motor de ruteo de dos áreas operativas muy distintas de la empresa:

1. **Escenario A (Rutas troncales nacionales):** Un camión blindado que debe visitar exactamente **18 bóvedas bancarias** regionales y regresar a la central con garantía matemática del **100% de optimalidad** en menos de 1 segundo.
2. **Escenario B (Flota de última milla):** Un repartidor que debe entregar **1,200 paquetes** en un día dentro de la zona metropolitana, donde el mapa debe recalcularse en menos de **3 segundos**.

Entregables de la Actividad

Sin necesidad de programar aún el código final, elabora un **Reporte Técnico de Decisión Algorítmica (2 cuartillas)** que responda y justifique:

1. **Elección de Enfoque:** ¿Qué algoritmo asignarías al **Escenario A** y cuál al **Escenario B**? *(Opciones a evaluar: Fuerza Bruta, Branch & Bound, Held-Karp con Bitmask, Vecino Más Cercano + 2-Opt, o Algoritmo de Christofides)*.
2. **Justificación Matemática:** Calcula el número aproximado de operaciones que tomaría tu algoritmo elegido en cada escenario frente a los que descartaste.
3. **Diseño de Estructuras de Datos:** Si tuvieras que implementarlo la próxima semana, ¿qué estructuras de datos usarías (ej. cola de prioridad `heapq`, matriz con máscara de bits `1 << n`, lista de adyacencia) y cuál sería su consumo de memoria RAM?